

List of Practical :

1. Familiarization with Raspberry Pi and perform necessary software installation.

Answer:

1. Download and Install Raspberry Pi Imager 2.0

Go to the official website (raspberrypi.com/software) and download the latest version of Raspberry Pi Imager for your operating system (Windows, macOS, or Linux).

Run the installer, follow the on-screen prompts, and launch the program.

2. Prepare Your SD Card

Insert a microSD card (minimum 32GB, Class 10 or faster recommended).

For best performance, use official Raspberry Pi cards, SanDisk Ultra, or SanDisk Extreme.

3. Launch the Imager and Start the Wizard

Open Raspberry Pi Imager 2.0. The new interface guides you through each step sequentially.

4. Select Your Raspberry Pi Device

Choose the exact model you are using (e.g., Raspberry Pi 5, Raspberry Pi 4, etc.). This helps the tool apply the correct default settings.

5. Choose the Operating System

Select **Raspberry Pi OS**.

- Choose the **64-bit** version for better performance (recommended for Raspberry Pi 4 and 5).
- Choose the **32-bit** version for maximum software compatibility on older models.

You can also select Lite versions or other operating systems if needed.

6. Select the Storage Device (SD Card)

Choose your inserted microSD card from the list.

Important: Double-check that you have selected the SD card and not your computer's hard drive. Enable the "exclude system drives" option if available to prevent mistakes.

All data on the selected card will be erased.

7. Configure System Settings (New Feature)

Complete the following configuration steps in the wizard:

- **Set Location & Language:** Choose your country, language, and time zone.
- **Set Hostname:** Give your Raspberry Pi a custom network name (example: raspberrypi-desk).
- **Create User Account:** Set a new username and a strong password. (There is no longer a default “pi” user.)
- **Configure Wi-Fi:** Enter your wireless network name (SSID) and password so the Pi connects automatically on first boot.
- **Enable Remote Access:** Turn on SSH for command-line remote access. Choose password or public-key authentication.
- **Set Up Raspberry Pi Connect:** Create or sign in to a Raspberry Pi account to enable easy cloud-based remote desktop access without port forwarding.

8. **Review and Write the Image**

Review all your settings.

Click **Write**. The Imager will download the OS (if needed), write it to the SD card, and verify the data. This may take several minutes.

9. **Complete the Process**

Then the writing

2. Run python program on Pi having problem statement: word and character count of a given string.

Answer: **Python Program**

```
# Word and Character Count Program
```

```
text = input("Enter a string: ")
```

```
# Count words
```

```
word_count = len(text.split())
```

```
char_count = len(text)
```

```
print("Number of Words:", word_count)
```

```
print("Number of Characters:", char_count)
```

Sample Output

Enter a string: Welcome to Raspberry Pi

Number of Words: 4

Number of Characters: 23

To Run on Raspberry Pi

1. Open **Terminal**.
2. Create a file:
3. nano word_char_count.py
4. Paste the code and save (Ctrl + X, Y, Enter).
5. Run the program:

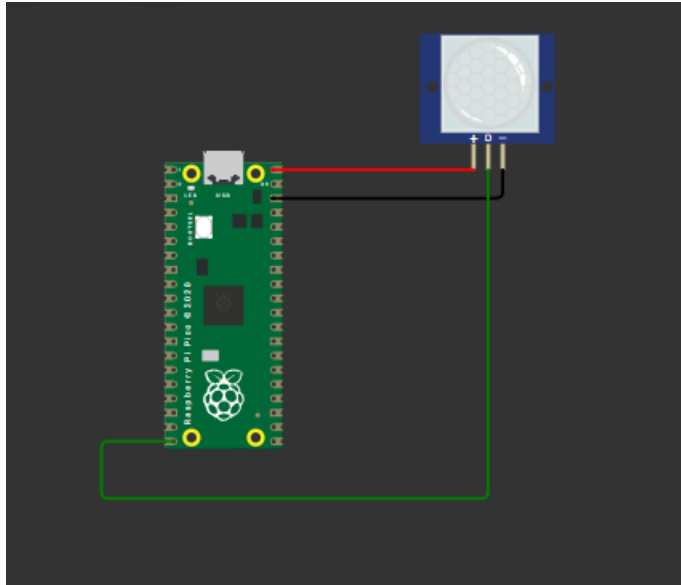
```
python3 word_char_count.py
```

3. Exercise on working principle of Raspberry Pi.with 40 PIN Interface component.

Reference Link: <https://wokwi.com/projects/393048863616453633>

Solution:

```
void setup() {  
  pinMode(15,INPUT);  
  Serial1.begin(115200);  
  Serial1.println("Hello, Raspberry Pi Pico!");  
}  
  
void loop(){  
  int pir = digitalRead(15);  
  if(pir == HIGH)  
  {  
    Serial1.println("MOVEMENT DETECTED");  
    delay(500);  
  }  
  else  
  {  
    Serial1.println("NO MOVEMENT DETECTED");  
    delay(500);  
  }  
}
```



4. To interface Bluetooth with Raspberry Pi and write a program to turn LED ON/OFF when 1/0 is received form smartphone using Bluetooth

Note: Required Physical Device

Reference Link: <https://iot-amrt.vlabs.ac.in/exp/bluetooth-led-control-pi/index.html>

Connection

- Connect VCC of HC-05 Bluetooth module to VBUS (5V) of Raspberry Pi Pico.
- Connect TXD of HC-05 to GP0 of Raspberry Pi Pico.
- Connect RXD of HC-05 to GP1 of Raspberry Pi Pico.
- Connect GND of HC-05 to GND of Raspberry Pi Pico.
- Connect the positive terminal of the LED to the resistor.
- Connect the other terminal of the resistor to GP19 of Raspberry Pi Pico.
- Connect the negative terminal of the LED to GND of Raspberry Pi Pico.
- Click the Bluetooth icon on the smartphone to enable Bluetooth.
- Turn on Bluetooth by clicking the switch on the smartphone screen.
- Select Raspberry Pi from available devices to pair.
- After pairing, use the smartphone buttons to turn the LED on or off.

Python Program:

```
from machine import Pin, UART
```

```
uart = UART(0, 9600)
```

```
led = Pin(19, Pin.OUT)
```

```
while True:
```

```
    if uart.any():
```

```
        command = uart.readline()
```

```
        if command == b'ON':
```

```
            led.high()
```

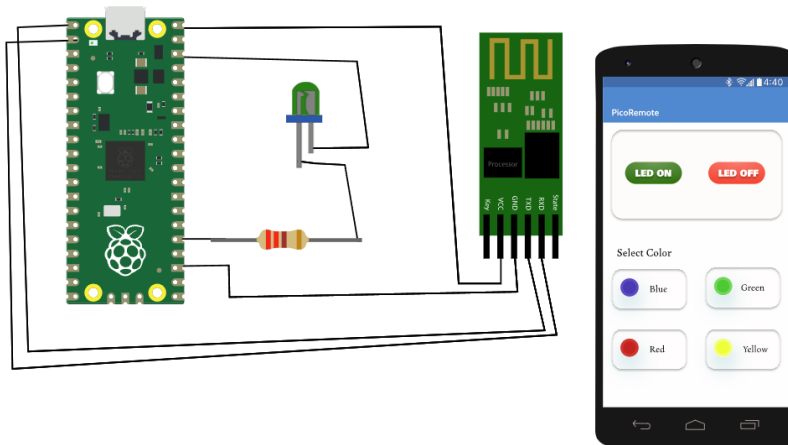
```
            print("LED ON")
```

```
        elif command == b'OFF':
```

```
            led.low()
```

```
            print("LED OFF")
```

Devices: Raspberry Pi Pico, HC-05, Resistor, LED, Smartphone



5. Connect with the Available Wi-Fi Using Arduino

Note: Required Physical Device

Reference Link: <https://wokwi.com/projects/356552645200576513>

```
#include <WiFi.h>

// Replace with your network credentials (STATION)

#define ssid "Wokwi-GUEST"

#define password ""

void initWiFi() {

  WiFi.mode(WIFI_STA);

  WiFi.begin(ssid, password);

  // WiFi.begin("Wokwi-GUEST", "");

  Serial.print("Connecting to WiFi ..");

  while (WiFi.status() != WL_CONNECTED) {

    Serial.println(WiFi.status());

    Serial.print('.');

    delay(1000);

  }

  Serial.println("Connected");

  Serial.println(WiFi.status());

  Serial.println(WiFi.localIP());

  Serial.print("RRSI: ");
```

```
    Serial.println(WiFi.RSSI());  
}
```

```
void setup() {  
    Serial.begin(115200);  
    initWiFi();  
}
```

```
void loop() {  
    // put your main code here, to run repeatedly:  
}
```

```
load:0x40080400,len:2972  
entry 0x400805dc  
Connecting to WiFi ..6  
.0  
.0  
.0  
.Connected  
3  
10.10.0.2  
RSSI: -93
```


6. Write a program on Raspberry Pi to retrieve temperature and humidity data from thingspeak cloud

New: <https://wokwi.com/projects/469089414257926145>

```
import network, urequests, time
```

```
# ----- WiFi -----
```

```
wlan = network.WLAN(network.STA_IF)
```

```
wlan.active(True)
```

```
wlan.connect("Wokwi-GUEST", "")
```

```
while not wlan.isconnected():
```

```
    time.sleep(0.5)
```

```
print("WiFi connected\n")
```

```
# ----- ThingSpeak settings -----
```

```
# Public demo channel (works out of the box, no key needed)
```

```
CHANNEL_ID = "12397"
```

```
READ_KEY = ""
```

```
TEMP_FIELD = "field4" # temperature on demo channel
```

```
HUM_FIELD = "field3" # humidity on demo channel
```

```
# ----- Build the READ url (do not change) -----
```

```
URL = "https://api.thingspeak.com/channels/" + CHANNEL_ID + "/feeds/last.json"
```

```
if READ_KEY:
```

```
    URL += "?api_key=" + READ_KEY
```

```
# ----- Main loop -----
```

```
while True:

    try:

        r = urequests.get(URL)

        data = r.json()

        r.close()

    if isinstance(data, dict):

        print("Time  :", data.get("created_at"))

        print("Temp  :", data.get(TEMP_FIELD))

        print("Humidity:", data.get(HUM_FIELD))

        print("-" * 30)

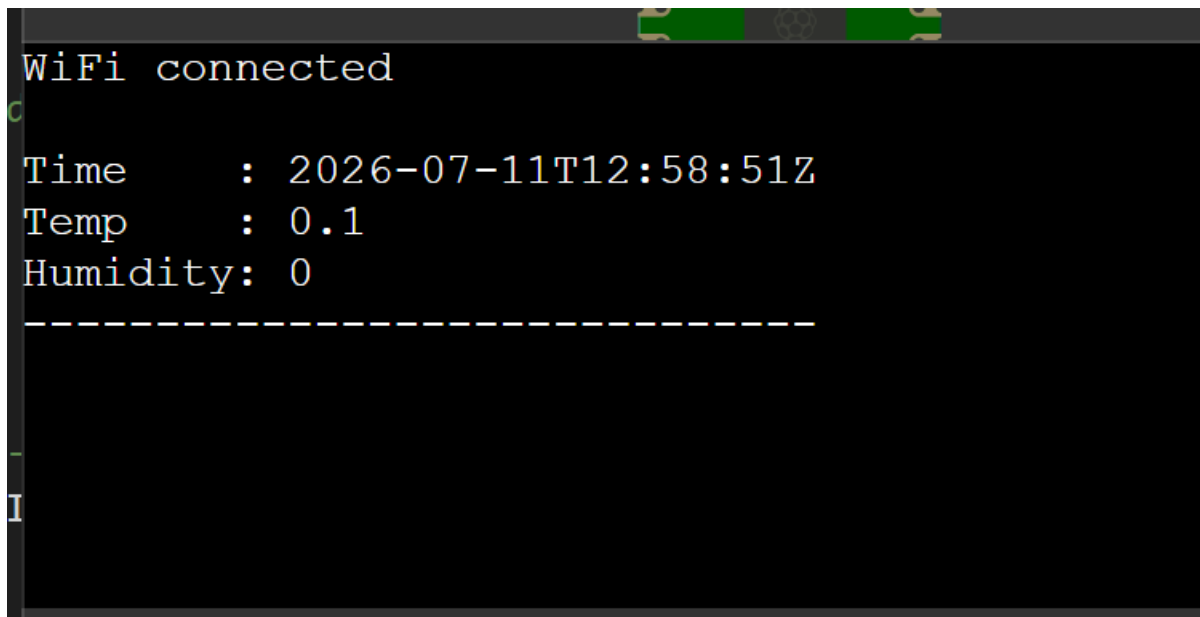
    else:

        print("Bad channel ID / key, server said:", data)

except Exception as e:

    print("Network error:", e)

time.sleep(20)
```



7. Write a program on Raspberry Pi to publish temperature data to MQTT broker.

Reference Link: <https://wokwi.com/projects/322577683855704658>

```
import network
```

```
import time
```

```
from machine import Pin
```

```
import dht
```

```
import ujson
```

```
from umqtt.simple import MQTTClient
```

```
# MQTT Server Parameters
```

```
MQTT_CLIENT_ID = "micropython-weather-demo"
```

```
MQTT_BROKER = "broker.mqttdashboard.com"
```

```
MQTT_USER = ""
```

```
MQTT_PASSWORD = ""
```

```
MQTT_TOPIC = "wokwi-weather"
```

```
sensor = dht.DHT22(Pin(15))
```

```
print("Connecting to WiFi", end="")
```

```
sta_if = network.WLAN(network.STA_IF)
```

```
sta_if.active(True)
```

```
sta_if.connect('Wokwi-GUEST', '')
```

```
while not sta_if.isconnected():
```

```
    print(".", end="")
```

```
    time.sleep(0.1)
```

```
print(" Connected!")
```

```
print("Connecting to MQTT server... ", end="")
```

```
client = MQTTClient(MQTT_CLIENT_ID, MQTT_BROKER, user=MQTT_USER,  
password=MQTT_PASSWORD)
```

```
client.connect()
```

```
print("Connected!")
```

```
prev_weather = ""
```

```
while True:
```

```
    print("Measuring weather conditions... ", end="")
```

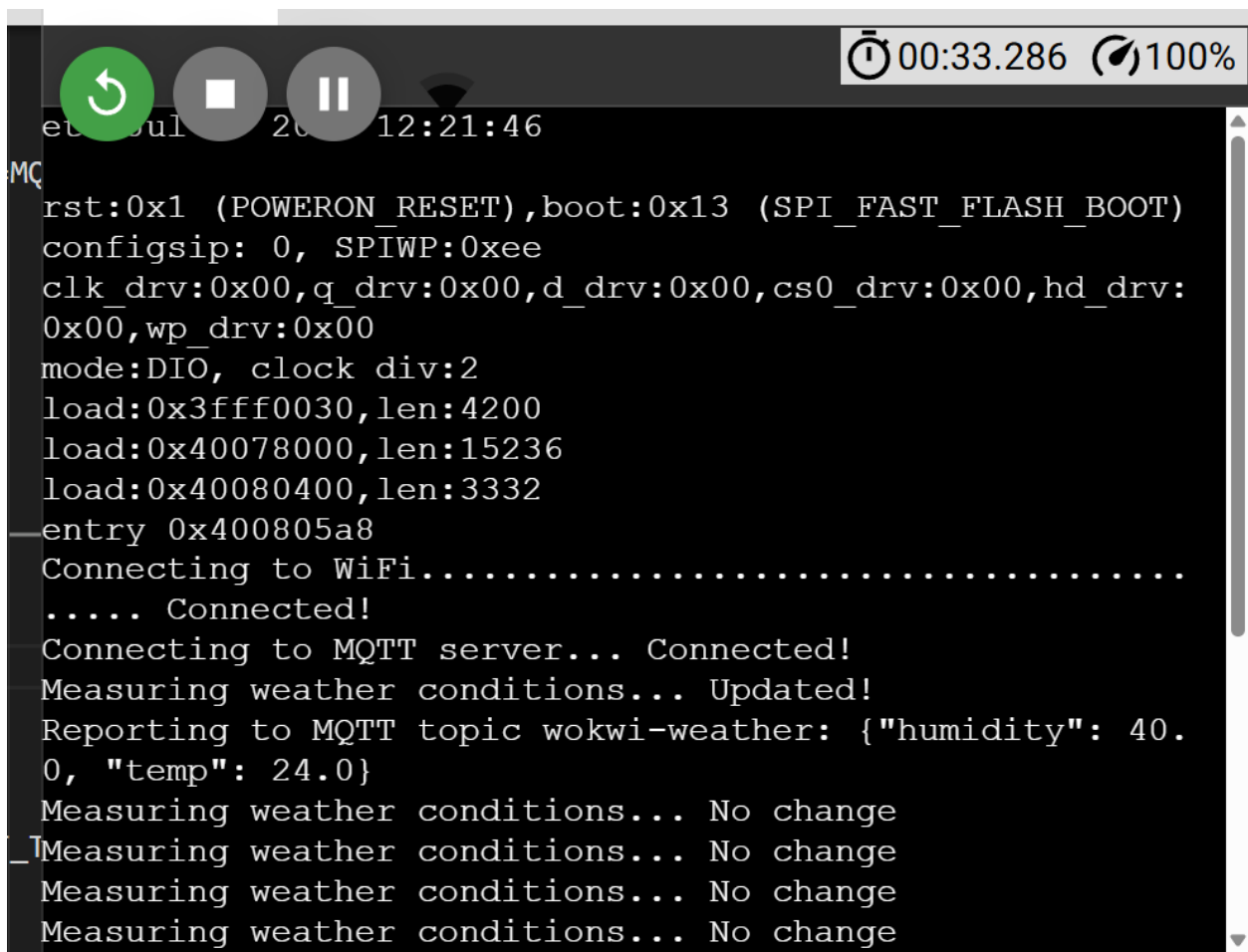
```
    sensor.measure()
```

```
    message = ujson.dumps({
```

```
        "temp": sensor.temperature(),
```

```
        "humidity": sensor.humidity(),
```

```
})  
  
if message != prev_weather:  
    print("Updated!")  
    print("Reporting to MQTT topic {}: {}".format(MQTT_TOPIC, message))  
    client.publish(MQTT_TOPIC, message)  
    prev_weather = message  
  
else:  
    print("No change")  
  
time.sleep(1)
```



```
et Jul 20 12:21:46  
MQ  
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)  
configsip: 0, SPIWP:0xee  
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:  
0x00,wp_drv:0x00  
mode:DIO, clock div:2  
load:0x3fff0030,len:4200  
load:0x40078000,len:15236  
load:0x40080400,len:3332  
entry 0x400805a8  
Connecting to WiFi.....  
..... Connected!  
Connecting to MQTT server... Connected!  
Measuring weather conditions... Updated!  
Reporting to MQTT topic wokwi-weather: {"humidity": 40.  
0, "temp": 24.0}  
Measuring weather conditions... No change  
Measuring weather conditions... No change  
Measuring weather conditions... No change  
Measuring weather conditions... No change
```

8. Connect Raspberry Pi with your existing system components.

Reference Link: <https://wokwi.com/projects/357627199664450561>

Project objective: To test a passive buzzer to play an alarm sound at one second interval

#

Hardware and connections used:

Passive buzzer GND to Raspberry Pi Pico GND

Passive buzzer + Pin to GPIO Pin 15

#

Programmer: Adrian Josele G. Quional

if passive buzzer is used, import the Speaker class from picozero

from picozero import Speaker

from time import sleep

creating a Speaker object

speaker = Speaker(15)

continuously beep at 1 sec interval while the board has power

note: a passive buzzer can also be used to play different tones

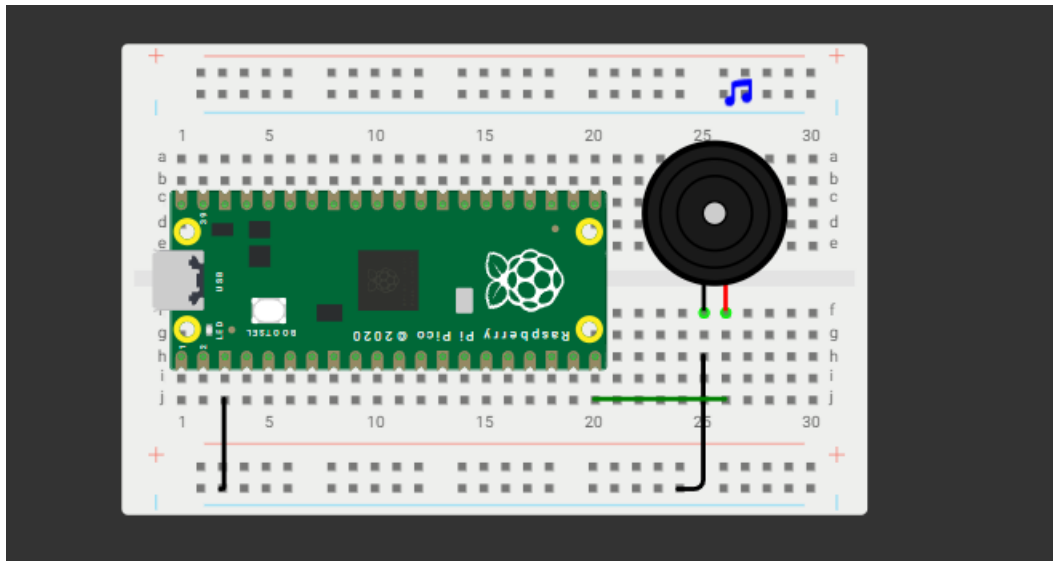
while True:

 speaker.on()

 sleep(1)

 speaker.off()

 sleep(1)



9. IoT based DC motor speed control using Arduino/Raspberry Pi.

Reference Link: <https://wokwi.com/projects/462024572818446337>

from machine import Pin, PWM, ADC

import time

1. DC Motor Setup (GP15)

dc_motor = PWM(Pin(15))

dc_motor.freq(1000)

2. Servo Setup (GP16)

servo = PWM(Pin(16))

servo.freq(50) # Servos require exactly 50Hz

3. Potentiometer (GP26)

pot = ADC(26)

```

# Servo pulse helper

def set_servo_duty(angle):
    # Map 0-180 degrees to ~1638-8192 duty (0.5ms to 2.5ms)
    duty = int(((angle / 180) * 6554) + 1638)
    servo.duty_u16(duty)

current_angle = 0
step_direction = 1

while True:
    # Read Potentiometer
    pot_val = pot.read_u16()

    # 1. Update DC Motor Speed
    dc_motor.duty_u16(pot_val)

    # 2. Calculate Servo "Speed" (Delay)
    # Mapping pot to a delay between 0.001s (Fast) and 0.05s (Slow)
    # Note: If pot is 0, we stop the servo
    speed_delay = (65535 - pot_val) / 1000000 + 0.001

    if pot_val > 1000:
        # Move servo one step
        current_angle += step_direction

        if current_angle >= 180 or current_angle <= 0:

```



```

step_direction *= -1 # Reverse direction at boundaries

set_servo_duty(current_angle)

speed_percent = int((pot_val / 65535) * 100)

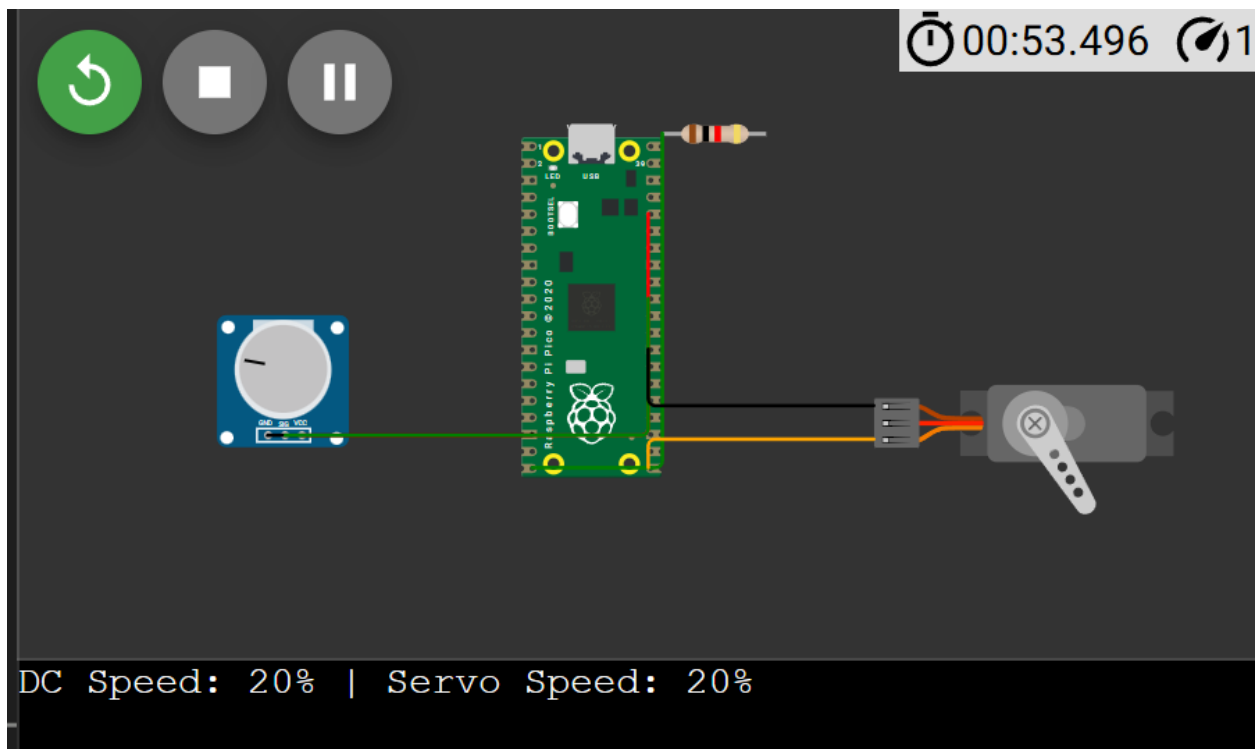
print(f"DC Speed: {speed_percent}% | Servo Speed: {speed_percent}%", end="\r")

else:

    print("Motors Stopped...      ", end="\r")

time.sleep(speed_delay)

```



10. To study of IoT Data Logging using Beaglebone Black and Thingspeak

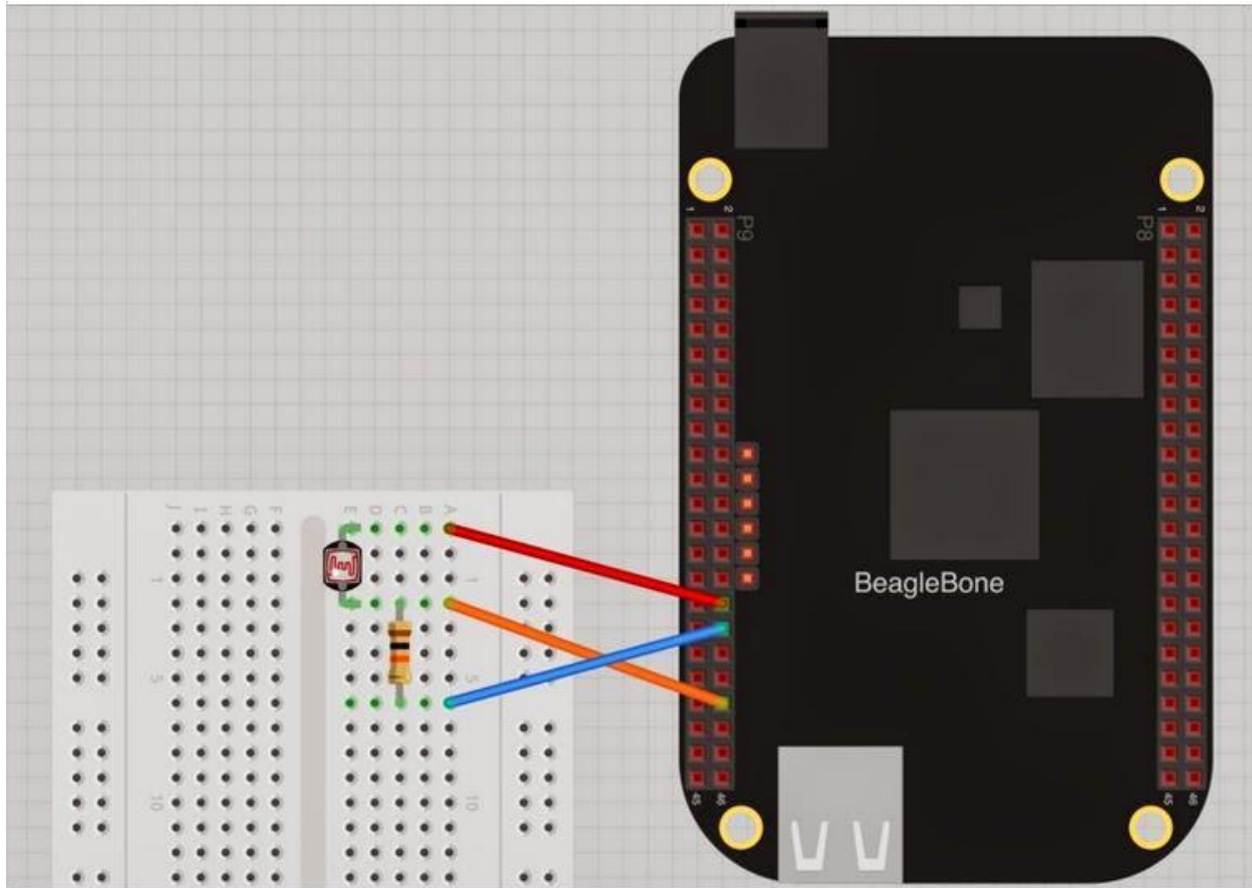
Reference Link: <https://wokwi.com/projects/460072555056502785>

Hardware Required

Sr. No.	Component	Quantity
1	BeagleBone Black	1
2	USB Cable	1
3	Breadboard	1
4	Light Dependent Resistor (LDR)	1
5	10 kΩ Resistor	1
6	Connecting Wires	As required
7	PC/Laptop with Internet	1

Software Required

- Debian Linux on BeagleBone Black
- Python
- Adafruit_BBIO Library
- Internet Connection
- ThingSpeak Account
- SSH Terminal (PuTTY)



Algorithm

1. Start the BeagleBone Black.
2. Connect BBB to the Internet.
3. Create a ThingSpeak account and a new channel.
4. Note the **Write API Key**.
5. Connect the LDR circuit to the analog input.
6. Initialize the ADC using the Adafruit_BBIO library.
7. Read the analog sensor value.
8. Convert the ADC reading into voltage.
9. Send the voltage value to ThingSpeak using an HTTP POST request.
10. Wait for approximately 15–20 seconds before sending the next reading.
11. Repeat the process continuously.

12. Observe the real-time graph on ThingSpeak.

Program (Python)

```
import Adafruit_BBIO.ADC as ADC
import time
import urllib, urllib

sensor_pin = "P9_40"

ADC.setup()

while True:

    reading = ADC.read(sensor_pin)
    voltage = reading * 1.8

    params = urllib.urlencode({
        'field1': voltage,
        'key': 'YOUR_WRITE_API_KEY'
    })

    headers = {
        "Content-type": "application/x-www-form-urlencoded",
        "Accept": "text/plain"
    }
```

```
conn = httplib.HTTPConnection("api.thingspeak.com:80")
```

```
conn.request("POST", "/update", params, headers)
```

```
response = conn.getresponse()
```

```
print("ADC Reading :", reading)
```

```
print("Voltage :", voltage)
```

```
time.sleep(16)
```

Replace **YOUR_WRITE_API_KEY** with the Write API Key obtained from your ThingSpeak channel. (noobtechiespeaks.blogspot.com)

Working

1. The LDR senses the surrounding light intensity.
2. The voltage across the LDR changes according to light intensity.
3. BeagleBone Black reads this voltage through its Analog-to-Digital Converter (ADC).
4. The Python program converts the ADC value into voltage.
5. The measured value is uploaded to the ThingSpeak cloud using an HTTP POST request.
6. ThingSpeak stores the data and automatically generates graphs for visualization.
7. The graph updates every 15–20 seconds with new sensor readings.
(noobtechiespeaks.blogspot.com)

Sample Output

ADC Reading : 0.412

Voltage : 0.7416 V

HTTP Response : 200 OK

Data uploaded successfully.

Result

The analog sensor data was successfully acquired using **BeagleBone Black** and uploaded to the **ThingSpeak cloud platform**. The sensor readings were stored and displayed as real-time graphs, demonstrating successful IoT-based cloud data logging.